

<HLS Programming with FPGAs>

The HLS Book

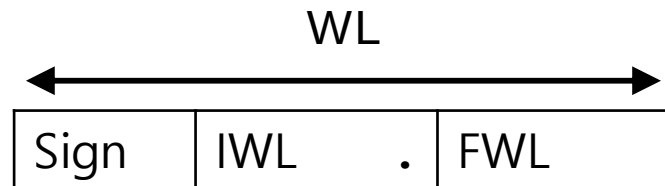
Chapter 3

CORDIC 3 and Fixed-Point Optimization

Choi, Young-kyu

Recap: Fixed-Point Representation

- A number with a fixed number of digits before/after the radix point
- Format:



S = sign bit

WL = word length

IWL = integer word length

FWL = fractional word length

Learning Objectives

- Perform simulation-based fixed-point optimization for CORDIC

Baseline Implementation Specification

- Number of CORDIC iterations : 5
- Variable type: float (single-precision)
- Input data : 10000 values in range of $-\pi/2 \sim \pi/2$
- Output data : cosine and sine value of input data
- Output RMSE: 0.0245
- Output maximum error: 0.0627
- Resource consumption:

BRAM	DSP	FF	LUT	URAM
0	10 1425	975	0	
-	-	-	-	-

Optimization Goal

- Given target error threshold (e.g., RMSE, max error, SQNR)
- Minimize resource consumption (e.g., LUT, FF, DSP, area)

Step 1: Measure Dynamic Range

- Step 1-1) Build a reference implementation in floating-point (preferably in double precision)
- Step 1-2) Insert **code** to measure minimum and maximum value of each variables
 - Ex)

```
theta = theta - sigma * cordic_phase[j]; //ref  
    theta_max = max(theta_max, theta);      //inserted  
    theta_min = min(theta_min, theta);      //inserted
```
- Step 1-3) Run C simulation
- Step 1-4) Obtain minimum and maximum value of each variables
- Shortcut: Use domain knowledge (e.g. range of cos)

Step 2: Set Initial IWL/FWL

- For all variables and constants
 - Step 2-1) Set sign and IWL to the value measured in step 1
 - Step 2-2) Set initial WL to 16b ~ 32b (ap_int<16> ~ ap_int<32>)
 - Rule of thumb : 1bit leads to about 6dB of SQNR
 - Step 2-3) Set FWL = WL - IWL
 - Ex) if a variable has dynamic range of 4.2 ~ 1.6 (WL=16b)
 - sign : 0b, IWL : 3b, FWL : 13b (= 16-3)
 - Ex) if a variable has dynamic range of 3.6 ~ -7.1 (WL=16b)
 - sign : 1b, IWL : 3b, FWL : 12b (= 16-3-1)

Step 3 : Shift & Sign Extend for Operations

- Addition

- Ex) $C = A$ (sign 1b, IWL 3b, FWL 4b) + B (sign 1b, IWL 5b, FWL 2b)

- (from step 1) Measure the dynamic range of variable C measured through simulation – ex) C 's sign=1b, IWL=5b

- (step 3-1) Align FWL & sign extend

A :	S	S	S	X	X	X	.	X	X	X	X
+ B :	S	X	X	X	X	X	.	X	X	0	0
= C :	S	X	X	X	X	X	.	X	X	X	X

-> $\text{ap_int}\langle 10 \rangle C = ((\text{ap_int}\langle 10 \rangle) A) + (((\text{ap_int}\langle 10 \rangle) B) \ll 2);$

(C : sign 1b, IWL 5b, FWL 4b)

- (step 3-2, optional) Rounding

- Round down

-> `ap_int<8> C = (((ap_int<8>) A)>>2) + ((ap_int<8>)B);`
(C : sign 1b, IWL 5b, FWL 2b)

Warning : shift (<<,>>) has lower precedence than addition (+) in C → always put **parenthesis** around bitwise shift operators

- Multiplication

- Ex) $C = A$ (sign 0b, IWL 3b, FWL 4b) * B (sign 0b, IWL 5b, FWL 2b)

- (from step 1) Measure the dynamic range of variable C measured through simulation – ex) C 's IWL=7b, no sign bit

- (step 3-1) Compute C 's FWL ($4b + 2b = 6b$) and WL ($0b + 7b + 6b = 13b$)

- (step 3-2) Sign extend

- `ap_uint<13> C = ((ap_uint<13>)A) * ((ap_uint<13>)B);`

- (step 3-3, optional) Rounding

- Quiz: What should be WL of variable C?
 - $C = A * B;$ // A: sign 1b, IWL 3b, FWL 4b
 // B : sign 1b, IWL 2b, FWL 5b
 // C : ? IWL 4b ? - no rounding

$$\begin{aligned}
 &(\text{sign } 1\text{b, IWL } 3\text{b, FWL } 4\text{b}) * (\text{sign } 1\text{b, IWL } 2\text{b, FWL } 5\text{b}) \\
 &= (\text{sign } \cancel{2}\text{b, IWL } \cancel{5}\text{b, FWL } 9\text{b}) \\
 &\qquad\qquad\qquad 1 \qquad\qquad 4
 \end{aligned}$$

- Answer:
 - 14b (sign 1b, IWL 4b, FWL 9b)

Exercise

- Suppose
 - A : sign : 1b, IWL : 5b, FWL : 8b (ap_int<14>)
 - B : sign : 0b, IWL : 7b, FWL : 1b (ap_uint<8>)
 - C : sign : 1b, IWL : 7b, FWL : 8b (ap_int<16>)
- Shift & sign extend the below computation (no rounding) and write down the HLS C code :
 - $C = A + B;$

Solution to Exercise

- Suppose
 - A : sign : 1b, IWL : 5b, FWL : 8b (ap_int<14>)
 - B : sign : 0b, IWL : 7b, FWL : 1b (ap_uint<8>)
 - C : sign : 1b, IWL : 7b, FWL : 8b (ap_int<16>)
- Shift & sign extend the below computation (no rounding) and write down the HLS C code :
 - $C = A + B;$
- Solution:
 - $C = ((\text{ap_int}<16>) A) + (((\text{ap_int}<16>) B) \ll 7);$

Step 4: Measure Output Error

- Step 4-1) Run C simulation of your fixed-point implementation
- Step 4-2) Compute output error (e.g. RMSE, SQNR) against the reference implementation
- Step 4-3) Check if output error exceeds error threshold
 - If exceeded, stop fixed-point optimization
 - If not exceeded, go to step 5

Step 5: Reduce FWL

- Step 5-1) Choose one variable (or a group of variables) to reduce FWL
 - You may choose randomly
 - Or choose a variable that gives most resource reduction with least error increase (greedy choice)
 - Hint: You can group variables with similar FWL
 - Ex 1) $C = A + B$; $E = C + D$; // variables $A \sim E$ will probably have same FWL
 - Ex 2) `current_cos`, `current_sin`, `cos_shift`, `sin_shift` will probably have same FWL (but not necessarily `theta`)

```
current_cos = (theta < 0) ? current_cos + sin_shift : current_cos - sin_shift;  
current_sin = (theta < 0) ? current_sin - cos_shift : current_sin + cos_shift;
```

- Step 5-2) Reduce FWL of the chosen variable by 1
 - FWL reduction lowers hardware cost
- Step 5-3) Go back to step 4 and iterate until target error threshold has been reached

Please perform fixed-point optimization of
your CORDIC HLS implementation

Fixed-point Optimization Result

- Synthesis result:

- Baseline implementation (float):

BRAM	DSP	FF	LUT	URAM
0	10 1425	975	0	
-	-	-	-	-

- After optimization:

BRAM	DSP	FF	LUT	URAM
0	0 40	272	0	
-	-	-	-	-

- Error

- Baseline implementation (float):

- RMSE: 0.0245, max error: 0.0627

- After optimization:

- RMSE: 0.0256, max error: 0.0781

Discussions

- Simulation-based fixed-point optimization
 - Advantage:
 - Can perform aggressive optimization
 - Disadvantage:
 - Time-consuming
 - Not very robust against overflow – depends on dataset
- Analytical fixed-point optimization
 - Analyzes the quantization noise with statistical analysis of the code

Further Readings

- Simulation-based optimization
 - Kim, Seehyun, Ki-Il Kum, and Wonyong Sung. "Fixed-point optimization utility for C and C++ based digital signal processing programs." *IEEE Transactions on Circuits and Systems II: Analog and Digital Signal Processing* 45.11 (1998): 1455-1464.
- Analytical optimization
 - Menard, Daniel, Romuald Rocher, and Olivier Sentieys. "Analytical fixed-point accuracy evaluation in linear time-invariant systems." *IEEE Transactions on Circuits and Systems I: Regular Papers* 55.10 (2008): 3197-3208.

Summary

- Simulation-based fixed-point optimization steps:
 - Step 1: Measure dynamic range
 - Step 2: Set initial IWL/FWL
 - Step 3 : Shift & sign extend for operations
 - Step 4: Measure output error
 - Step 5: Reduce FWL